Universidad de San Carlos de Guatemala
Escuela de Ingeniería en Ciencias y Sistemas, Facultad de Ingeniería
Introducción a la programación y computación 2, 1er. Semestre 2026

PLATAFORMA DISTRIBUIDA DE AUTORIZACIÓN TRIBUTARIA: IMPLEMENTACIÓN DE UNA API REST EN C# Y .NET CORE PARA LA GESTIÓN DE FACTURACIÓN ELECTRÓNICA

202503236 – Elder Rodolfo Ché Mijangos

Resumen

El presente documento detalla el diseño e implementación de un sistema cliente-servidor orientado a la Superintendencia de Administración Tributaria (SAT) para la gestión automatizada de Documentos Tributarios Electrónicos (DTE). El estudio aborda la modernización de los sistemas fiscales mediante la construcción de una API REST desarrollada en C# bajo el entorno .NET Core, evidenciando su pertinencia en el contexto nacional para agilizar la validación de transacciones comerciales. A nivel técnico, el proyecto adopta el paradigma de Programación Orientada a Objetos (POO) para modelar entidades tributarias y aplica algoritmos matemáticos estrictos, como el Módulo 11, para la verificación de Números de Identificación Tributaria (NIT). El impacto principal recae en la optimización computacional y la reducción de errores de facturación, logrando una persistencia de datos eficiente mediante archivos JSON. Se concluye que la separación de responsabilidades entre el frontend (Razor Pages) y el backend (API) garantiza un sistema escalable, interoperable y de alta cohesión.

Palabras clave

API REST, C#, Módulo 11, JSON, Arquitectura Cliente-Servidor..

Abstract

This document details the design and implementation of a client-server system aimed at the Superintendencia de Administración Tributaria (SAT) for the automated management of Electronic Tributary Documents (DTE). The study addresses the modernization of fiscal systems through the construction of a REST API developed in C# under the .NET Core framework, demonstrating its relevance in the national context to expedite the validation of commercial transactions. On a technical level, the project adopts the Object-Oriented Programming (OOP) paradigm to model tax entities and applies strict mathematical algorithms, such as Modulo 11, for the verification of Tax Identification Numbers (NIT). The primary impact lies in computational optimization and the reduction of billing errors, achieving efficient data persistence through JSON files. It is concluded that the separation of concerns between the frontend (Razor Pages) and the backend (API) guarantees a

scalable, interoperable, and highly cohesive system.

Keywords
REST API, C#, Modulo 11, JSON, Client-Server Architecture.

Introducción

La digitalización de los procesos fiscales es un pilar fundamental para la transparencia y eficiencia económica de un país. En Guatemala, la transición hacia esquemas electrónicos exige infraestructuras de software capaces de procesar altos volúmenes de información en tiempo real. Este ensayo documenta la construcción de un motor de autorización tributaria mediante una API HTTP que estandariza la recepción, validación y almacenamiento de solicitudes de facturación. El propósito principal es demostrar cómo la aplicación rigurosa de la Programación Orientada a Objetos, combinada con algoritmos de validación deterministas (como el Módulo 11) y expresiones regulares, permite crear un servicio robusto y tolerante a fallos. A lo largo del documento se explorarán las decisiones arquitectónicas, el modelado de Tipos de Dato Abstracto (TDA) y los mecanismos de persistencia implementados, dando respuesta a la interrogante de cómo estructurar un backend independiente que pueda ser consumido por múltiples plataformas tecnológicas.

Desarrollo del tema

La solución de software desarrollada se fundamenta en una arquitectura distribuida que separa las responsabilidades de visualización y procesamiento, garantizando un acoplamiento débil y una alta cohesión.

a. Arquitectura Cliente-Servidor y Diseño de la API El núcleo del proyecto es un "Backend as a Service" (BaaS) construido sobre ASP.NET Core, el cual expone una serie de *endpoints* HTTP. Esta API RESTful actúa como el procesador central de reglas de negocio, recibiendo cargas útiles en formato JSON. La ventaja técnica de este enfoque es la interoperabilidad; el sistema no está atado a una interfaz gráfica específica, lo que permite que sea consumido tanto por el simulador web desarrollado en Razor Pages como por herramientas de prueba especializadas como Postman. Se configuraron políticas de Intercambio de Recursos de Origen Cruzado (CORS) para permitir la comunicación segura entre el puerto del cliente web y el servidor de la API.

b. Programación Orientada a Objetos y Modelado de Datos La abstracción del problema tributario se materializó mediante la POO en C#. Se definieron Tipos de Dato Abstracto (TDA) para representar los dominios del problema. Clases como Contribuyente y FacturaRequest encapsulan las propiedades de las entidades, mientras que la clase AutorizacionResponse maneja el estado de salida. La lógica de negocio dicta que el identificador de aprobación debe ser correlativo y único por día, estructurado bajo el formato alfanumérico estricto $yyyymmdd########$. Para lograr esto, se implementó un gestor de estado que encapsula el

reinicio diario y el relleno de ceros (padding) utilizando los métodos nativos de formateo de cadenas del lenguaje.

c. Algoritmos de Validación: Módulo 11 y Expresiones Regulares Para garantizar la integridad de los datos, el sistema implementa múltiples filtros antes de autorizar una factura. La extracción de fechas desde cadenas de texto sin formato estandarizado ("Guatemala, dd/mm/yyyy hh24:mi") se resolvió mediante Expresiones Regulares (Regex), permitiendo aislar los valores cronológicos independientemente de los caracteres adyacentes.

El aspecto algorítmico más complejo radica en la validación del NIT mediante el método de Módulo 11. Este algoritmo matemático previene la inserción de registros erróneos en el Registro Tributario Unificado (RTU). El flujo computacional consiste en iterar sobre los dígitos del NIT (excluyendo el verificador), multiplicarlos por su posición ponderada inversa, sumar los productos y aplicar la operación de módulo matemático. Si el residuo complementario coincide con el dígito verificador (o la letra 'K' en caso de residuo 10), el NIT es admitido.

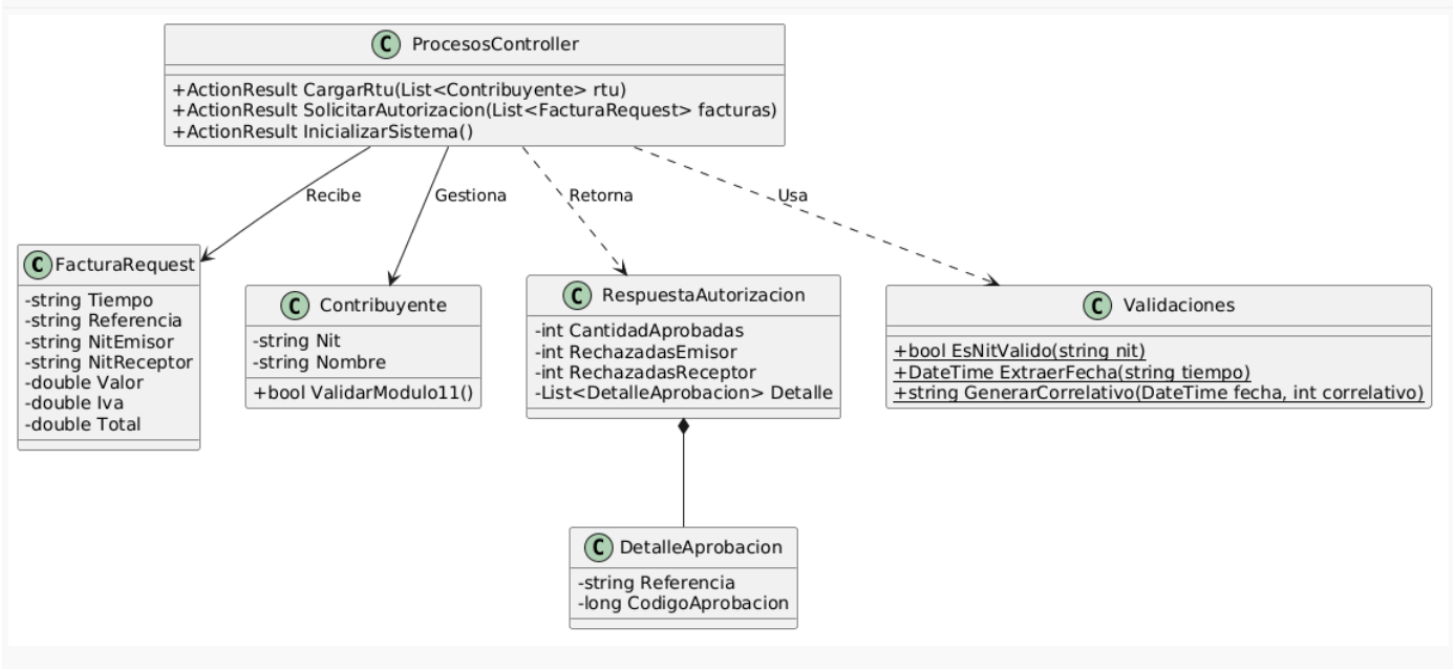
d. Persistencia y Generación de Reportes La capa de datos prescinde de motores de bases de datos relacionales en favor de un enfoque ligero de almacenamiento en archivos planos y estructuras JSON (base_rtu.json, registro_facturas.json). La manipulación de estos archivos se realiza a través de la librería System.Text.Json de C#. Para la generación de estadísticas complejas (Resumen de IVA y totales por fecha), se aprovechó el poder de *Language Integrated Query* (LINQ), aplicando funciones de agrupación (GroupBy) y sumatorias condicionales en memoria antes de devolver las respuestas HTTP, las cuales posteriormente son renderizadas dinámicamente utilizando Chart.js en el frontend.

Conclusiones

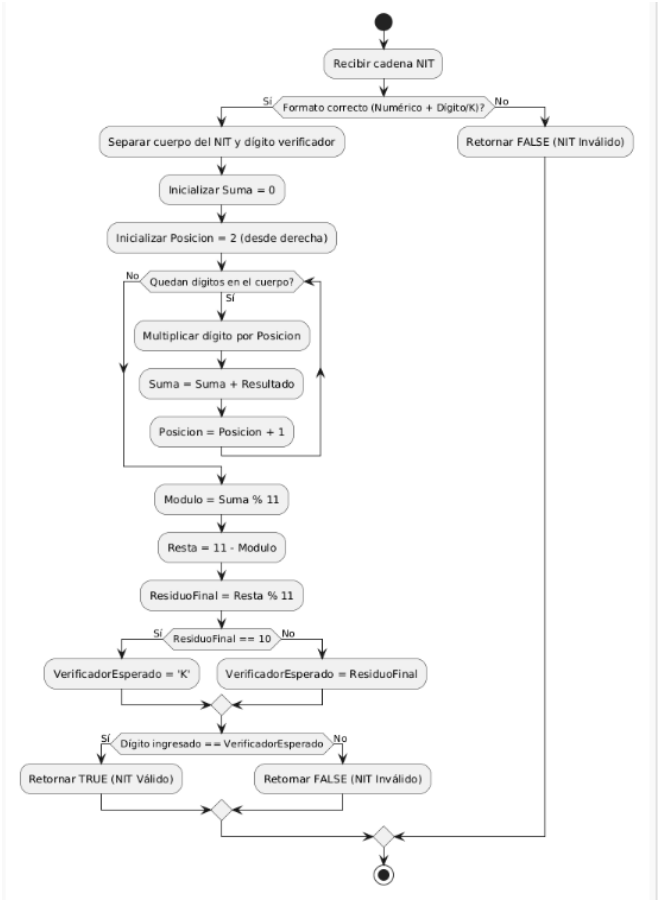
1. La implementación de una arquitectura cliente-servidor mediante una API REST en .NET Core demostró ser altamente eficaz, permitiendo centralizar la lógica de validación tributaria de manera independiente a la interfaz de usuario, garantizando así la escalabilidad del sistema.
2. El uso de Programación Orientada a Objetos y Tipos de Dato Abstracto facilitó la representación fiel de las entidades comerciales (Contribuyentes y Facturas), lo que redujo la complejidad del código y facilitó el mantenimiento y la extensión del software.
3. La integración algorítmica del Módulo 11 y expresiones regulares resultó indispensable para asegurar la integridad de los datos entrantes, mitigando errores humanos y rechazando transacciones fraudulentas en tiempo real antes de su persistencia en el almacenamiento local JSON.

Referencias bibliográficas

- Freeman, A. (2020). *Pro ASP.NET Core 3: Develop Cloud-Ready Web Applications Using MVC, Blazor, and Razor Pages*. Apress.
- Microsoft. (2025). *Documentación de C# y .NET Core*. Microsoft Learn. Recuperado de https://learn.microsoft.com/es-es/dotnet/csharp/
- Mozilla Developer Network [MDN]. (2025). *Expresiones Regulares (RegEx)*. Recuperado de https://developer.mozilla.org/es/docs/Web/JavaScript/Guide/Regular_expressions


ProcesosController
+ActionResult CargarRtu(List<Contribuyente> rtu)
+ActionResult SolicitarAutorizacion(List<FacturaRequest> facturas)
+ActionResult InicializarSistema()
Recibe
Gestiona
Retorna
Usa
FacturaRequest
-string Tiempo
-string Referencia
-string NitEmisor
-string NitReceptor
-double Valor
-double Iva
-double Total
Contribuyente
-string Nit
-string Nombre
+bool ValidarModulo11()
RespuestaAutorizacion
-int CantidadAprobadas
-int RechazadasEmisor
-int RechazadasReceptor
-List<DetalleAprobacion> Detalle
Validaciones
+bool EsNitValido(string nit)
+DateTime ExtraerFecha(string tiempo)
+string GenerarCorrelativo(DateTime fecha, int correlativo)
DetalleAprobacion
-string Referencia
-long CodigoAprobacion

Universidad de San Carlos de Guatemala
Escuela de Ingeniería en Ciencias y Sistemas, Facultad de Ingeniería
Introducción a la programación y computación 2, 1er. Semestre 2026



Universidad de San Carlos de Guatemala
Escuela de Ingeniería en Ciencias y Sistemas, Facultad de Ingeniería
Introducción a la programación y computación 2, 1er. Semestre 2026

